

Desenvolvimento de Serviços com Spring Boot

Etapa/Aula 05.1

Professor(a): Flávio Neves

E-mail: flavio.neves@prof.infnet.edu.br

Manipulação de Dados com JPA e Spring Boot

Roteiro da Aula

- BD e sua importância
- Manipulação de Dados com JPA e Spring Boot

Manipulação de Dados com JPA e Spring Boot

- Até aqui a gente já estudou alguns dos recursos do Spring Boot, porém nossas aplicações ainda não são capazes de armazenar os dados.

Manipulação de Dados com JPA e Spring Boot

- Vamos entender as tecnologias usadas para o armazenamento de dados usando JPA.
- Vamos explorar o processo de configuração do banco de dados.
- Fazer a inicialização do banco de dados.
- Acessar dados e gerenciar objetos de negócios no banco de dados com Spring Boot!

Spring Data

- O Spring Data é um conjunto de projetos do Spring para a manipulação de dados de diversas formas, entre elas:
 - Bancos de dados relacionais como o MySQL e o PostgreSQL; e
 - Bancos de dados NoSQL com o MongoDB e o Redis.
- A missão do Spring Data é fornecer um modelo de programação familiar e consistente, baseado no Spring, para acesso a dados, mantendo as características especiais do armazenamento de dados subjacente.

Spring Data

- Além do mapeamento, esse framework permite também o desenvolvimento de métodos para o acesso aos dados com pouco ou nenhum código sendo necessário para o seu desenvolvimento.
- Isso facilita muito o desenvolvimento das aplicações, e aumenta a produtividade dos programadores.

Spring Data JPA

- Um dos projetos do Spring Data é o Spring Data JPA para o desenvolvimento de aplicações com a Java Persistence API (JPA), que permite a implementação do mapeamento objeto-relacional de um modelo de dados.

Spring Data JPA

- Ele facilita o uso de tecnologias de acesso a dados, bancos de dados relacionais e não relacionais, estruturas de **map-reduce** e serviços de dados baseados em nuvem.
- Este é um projeto abrangente que contém muitos subprojetos específicos de uma determinada base de dados.
- Os projetos são desenvolvidos através do trabalho conjunto com muitas das empresas e programadores que estão por detrás destas tecnologias interessantes.

Spring Data JPA

- O Spring Data JPA, parte da grande família Spring Data, facilita a implementação de repositórios baseados em JPA (Java Persistence API)
- Facilita a criação de aplicações baseadas no Spring que utilizam tecnologias de acesso a dados.

Spring Data JPA

- A implementação de uma camada de acesso a dados para uma aplicação pode ser bastante complicada.
- É necessário escrever bastante código padrão para executar as consultas mais simples.
- Acrescente coisas como paginação, auditoria e outras opções frequentemente necessárias, e você acaba gastando muito esforço nessas tarefas.

Spring Data JPA

- O Spring Data JPA tem como objetivo melhorar significativamente a implementação de camadas de acesso a dados, reduzindo o esforço para que é realmente necessário.
- Como programador, escrever as interfaces do seu repositório utilizando qualquer número de técnicas, e o Spring liga-as automaticamente.
- Pode até utilizar localizadores personalizados ou consultas por exemplo e o Spring escreverá a consulta por si!

Spring Data JPA

- Spring Data (<https://spring.io/projects/spring-data>) permite acessar dados de uma variedade de fontes de dados:
 - Bancos de dados relacionais
 - Bancos de dados não relacionais,
 - Bancos de dados MapReduce e serviços de dados baseados em nuvem.
- Ele tenta fornecer um modelo de programação uniforme, fácil de usar e familiar por meio do Spring Framework.

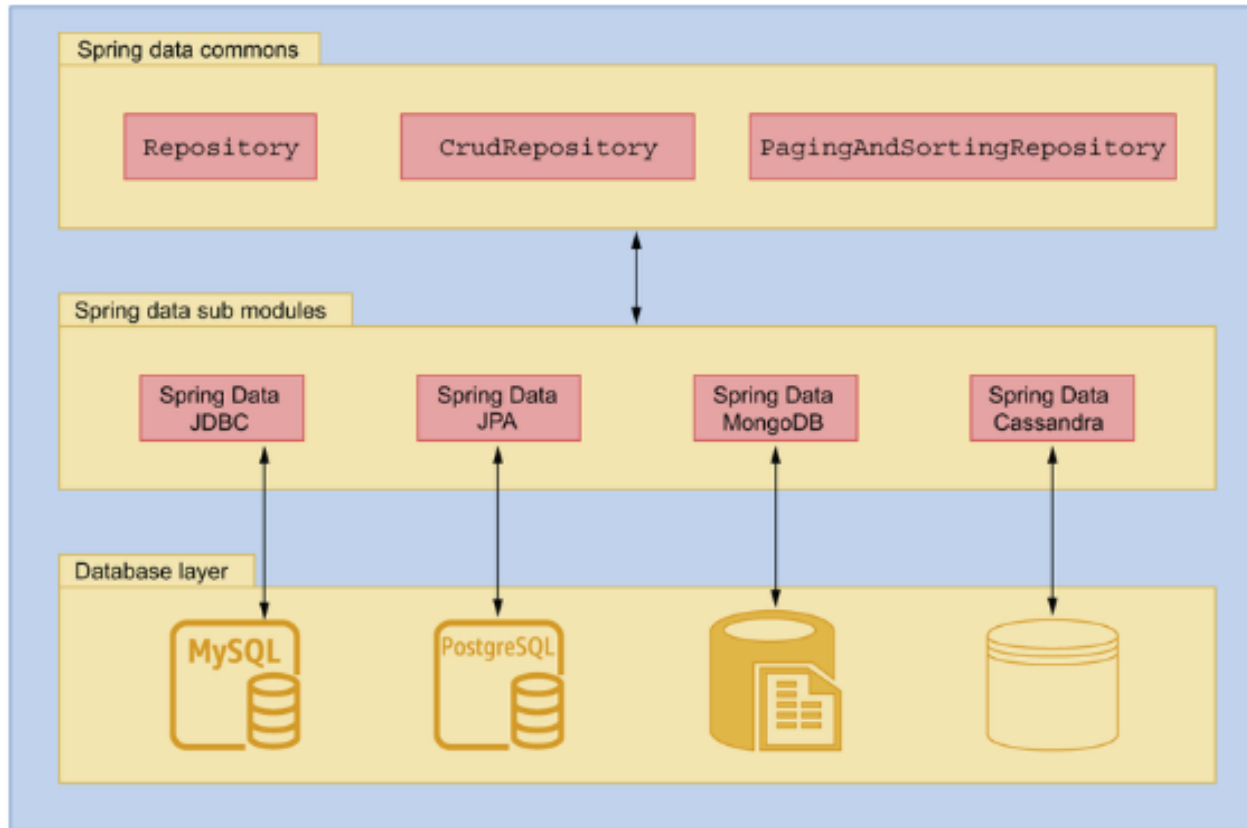
Spring Data JPA

- É um projeto guarda-chuva do Spring Framework que contém vários subprojetos, cada um deles direcionado a um banco de dados específico.
- Por exemplo, o módulo Spring Data JPA é específico para bancos de dados relacionais (por exemplo, H2, MySQL, PostgreSQL).
- Da mesma forma, Spring Data MongoDB visa fornecer suporte para o banco de dados MongoDB.

Spring Data JPA

Módulo	Propósito
Spring Data Commons	Ele contém os componentes básicos usados em todos os projetos Spring Data.
Spring Data JDBC	Este módulo fornece suporte de repositório para JDBC.
Spring Data JPA	Ele fornece suporte de repositório para JPA.
Spring Data MongoDB	Ele fornece suporte para banco de dados MongoDB baseado em documentos.
Spring Data REDIS	Ele fornece o suporte necessário para o armazenamento de dados Redis.
Spring Dados REST	Ele permite acessar repositórios de dados Spring como recursos REST.
Spring Data for Apache Cassandra	Este módulo fornece o suporte necessário para Apache Cassandra.

Spring Data JPA



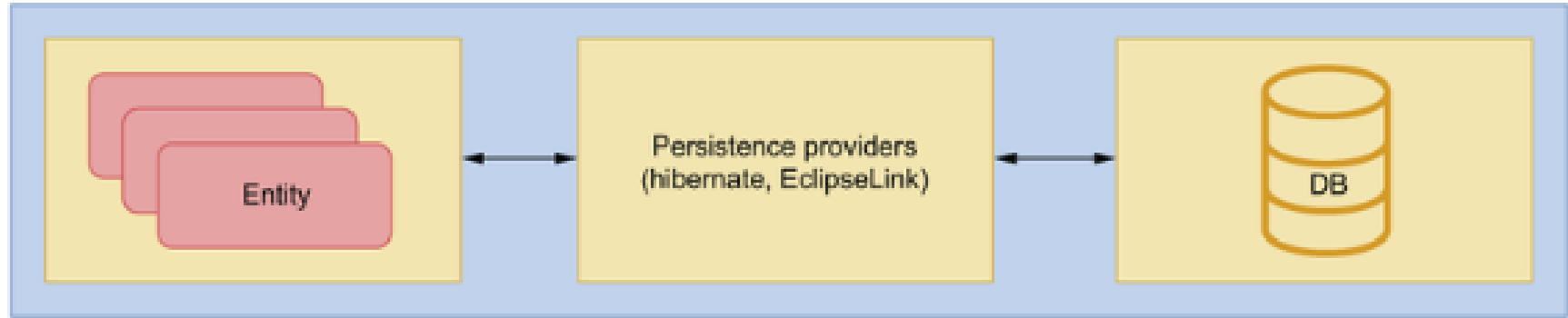
Spring Data JPA

- A maioria dos aplicativos no mundo atual precisa se comunicar com o banco de dados para armazenar e recuperar dados do aplicativo.
- E para conseguir essa interação, os desenvolvedores geralmente precisam escrever muitos códigos padronizados.
 - Por exemplo, na abordagem padrão Java Database Connectivity (JDBC), você precisa obter uma conexão de banco de dados,
 - Definir um arquivo PreparedStatement, definir as variáveis de ligação,
 - Executar a consulta e realizar o gerenciamento de recursos.

Spring JPA

- A Java Persistence API (JPA) elimina a maior parte desses encargos e fornece aos desenvolvedores uma ponte entre o modelo de objeto Java (por exemplo, objetos de negócios) e o modelo de banco de dados relacional (por exemplo, tabelas de banco de dados).
- Este mapeamento entre objetos Java e o modelo relacional é popularmente conhecido como mapeamento objeto-relacional (ORM).

Spring JPA

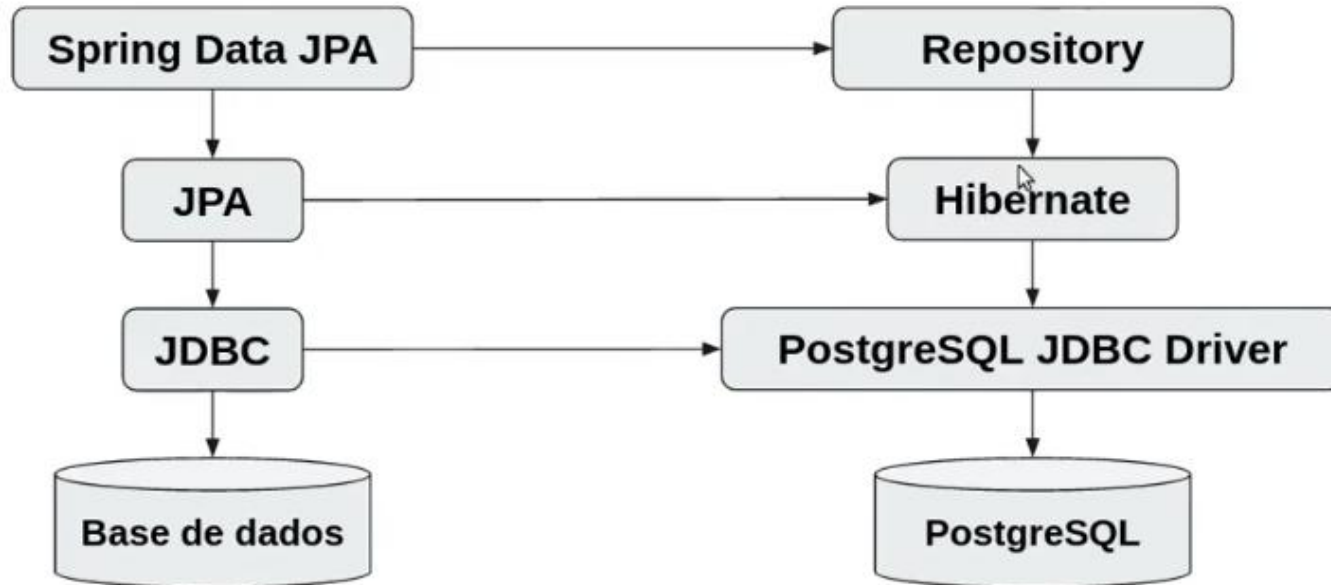


Uma visão geral do mapeamento objeto-relacional. Uma entidade representa um objeto de negócios a ser persistido. Um provedor de persistência implementa a especificação JPA.

Spring JPA

- JPA é uma especificação que fornece um conjunto de interfaces, classes e anotações para persistir e recuperar objetos de aplicativos de maneira fácil e concisa.
- Observe que é apenas uma especificação e descreve os padrões para as técnicas de ORM(Object-Relational Mapping).
- Existem vários fornecedores terceirizados, como Hibernate (<https://hibernate.org/orm/>) e EclipseLink (<https://www.eclipse.org/eclipselink/#jpa>) que fornecem uma implementação concreta desta especificação.

Spring Data JPA



JPA

- JPA: Java Persistence API(JPA) é uma especificação para mapeamento objeto-relacional;
 - Usado para mapear classes para tabelas, e atributos para colunas em tabelas.
 - Fazer a persistência e recuperação de dados
- Alguns das principais implementações são:
 - Hibernate (Red Hat)
 - EclipseLink (Eclipse Foundation)
 - OpenJPA (Apache)

```
@Entity
public class Person {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
    // getters setters
}

public void save(Person person) {
    session.beginTransaction();
    session.save(person);
    session.getTransaction().commit();
}
```

JDBC

- **JDBC**: conjunto de classes e interfaces que fazem o envio de instruções SQL para qualquer banco de dados.
 - Ele é uma especificação, desta forma os fornecedores de Banco de Dados devem seguir essa especificação para fornecer um driver **JDBC** ao seu banco de dados.

Project ▾

- ▾ exemploapijava G:\Outros computadores\meu laptop\pr...
 - > .idea
 - > banco
 - ▾ exemploapijava
 - > .mvn
 - ▾ src
 - ▾ main
 - > java
 - ▾ resources
 - static
 - templates
 - application.properties**
 - > test
 - > target
 - .gitattributes

application.properties × pom.xml (exemploapijava) ExemploapijavaApplication.java

```
1  spring.application.name=exemplo-api-java
2
3  server.port = 8081
4  spring.h2.console.enabled=true
5  spring.h2.console.path=/h2
6
7  spring.datasource.url=jdbc:h2:file:./banco/teste
8  spring.datasource.username=sa
9  spring.datasource.password=
10 spring.datasource.driver-class-name=org.h2.Driver
11
12 spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
13 spring.jpa.hibernate.ddl-auto=create
14 spring.jpa.show-sql=true
15 spring.jpa.properties.hibernate.format_sql=true
```

```
1 <dependencies>
2   <dependency>
3     <groupId>org.springframework.boot</groupId>
4     <artifactId>spring-boot-starter-data-jpa</artifactId>
5   </dependency>
6   <dependency>
7     <groupId>org.springframework.boot</groupId>
8     <artifactId>spring-boot-starter-webflux</artifactId>
9   </dependency>
10
11   <dependency>
12     <groupId>com.h2database</groupId>
13     <artifactId>h2</artifactId>
14     <scope>runtime</scope>
15   </dependency>
16   <dependency>
17     <groupId>org.springframework.boot</groupId>
18     <artifactId>spring-boot-starter-test</artifactId>
19     <scope>test</scope>
20   </dependency>
21   <dependency>
22     <groupId>io.projectreactor</groupId>
23     <artifactId>reactor-test</artifactId>
24     <scope>test</scope>
25   </dependency>
26 </dependencies>
```



Desenvolvimento de Serviços com Spring Boot

Etapa/Aula 05.1

Professor(a): Flávio Neves

E-mail: flavio.neves@prof.infnet.edu.br